Robust Differential Dynamic Programming for Rocket Engine Control: A Complete Mathematical Framework from First Principles

Engine Design Team

January 24, 2026

Contents

1 Introduction **4**
1.1 Who Should Read This Document . . . 4
1.2 Document Structure . . . 4
1.3 System Overview . . . 4
1.4 Control Objective . . . 5

2 Background: Optimal Control Theory **5**
2.1 What is Optimal Control? . . . 5
2.2 The General Optimal Control Problem . . . 5
2.3 Discrete-Time Formulation . . . 6
2.4 What is Differential Dynamic Programming (DDP)? . . . 6
2.5 The Bellman Principle of Optimality . . . 6
2.6 Why DDP Works . . . 7

3 First Principles Physics **7**
3.1 Ideal Gas Law . . . 7
3.1.1 Derivation from Kinetic Theory . . . 7
3.2 Polytropic Process . . . 8
3.3 Compressible Gas Flow . . . 8
3.3.1 Critical Pressure Ratio . . . 8
3.3.2 Choked Flow . . . 8
3.3.3 Subsonic Flow . . . 9
3.4 Mass Conservation . . . 9
3.5 Ullage Volume Dynamics . . . 9

4 System Dynamics **9**
4.1 State Vector . . . 9
4.2 Control Vector . . . 10
4.3 Discrete-Time Dynamics . . . 10
4.3.1 COPV Gas Mass Update . . . 10
4.3.2 COPV Pressure . . . 10
4.3.3 Regulator Pressure . . . 10
4.3.4 Gas Flow from COPV to Regulator . . . 11
4.3.5 Gas Flow from Regulator to Tanks . . . 11
4.3.6 Ullage Volume Update . . . 11
4.3.7 Tank Gas Mass Update . . . 11

4.3.8 Tank Pressure . . . 11
4.3.9 Feed Line Pressure Dynamics . . . 11

5 Differential Dynamic Programming **12**
5.1 Problem Formulation . . . 12
5.2 Running Cost . . . 12
5.2.1 Thrust Tracking . . . 12
5.2.2 Mixture Ratio Tracking . . . 12
5.2.3 Gas Consumption . . . 12
5.3 Bellman Equation . . . 12
5.4 Linearization . . . 13
5.5 Backward Pass . . . 13
5.5.1 Cost Derivatives . . . 13
5.5.2 Q-Function . . . 13
5.5.3 Optimal Control Update . . . 13
5.5.4 Regularization . . . 14
5.5.5 Value Function Update . . . 14
5.6 Forward Pass . . . 14
5.6.1 Line Search . . . 14
5.6.2 Forward Rollout . . . 14

6 Robustification **14**
6.1 Uncertainty Inflation . . . 14

7 Constraints **14**
7.1 Soft Constraints . . . 14
7.2 Hard Constraints . . . 14

8 Numerical Implementation **15**
8.1 Why Finite Differences? . . . 15
8.2 Finite Differences: The Basic Idea . . . 15
8.3 Choosing ϵ . . . 15
8.4 Computing the Dynamics Jacobian $\mathbf{A}_k$ . . . 16
8.5 Computing the Control Jacobian $\mathbf{B}_k$ . . . 16
8.6 Computing Cost Derivatives . . . 17
8.7 Computing Hessians . . . 18
8.8 Cholesky Decomposition for Positive Definiteness . . . 18
8.9 Convergence Criteria . . . 19
8.10 Value Function Hessian Projection . . . 19

9 Complete Controller Logic Walkthrough **19**
9.1 High-Level Control Loop . . . 19
9.2 Step 1: Read Sensors . . . 20
9.3 Step 2: Build State Vector . . . 20
9.4 Step 3: Generate Reference . . . 21
9.5 Step 4: Solve DDP . . . 21
9.5.1 4.1: Initialize Control Sequence . . . 21
9.5.2 4.2: Forward Rollout . . . 22
9.5.3 4.3: Backward Pass . . . 22
9.5.4 4.4: Forward Line Search . . . 23
9.5.5 4.5: Iterate . . . 23
9.6 Step 5: Apply Control . . . 23

9.7 Step 6: Update Internal State . . . 24
9.8 Complete Example: One Control Step . . . 24

10 Detailed Dynamics Derivation **25**
10.1 Complete State Update Equations . . . 25
10.1.1 COPV Gas Mass and Pressure . . . 25
10.1.2 Regulator Pressure with Oscillations . . . 25
10.1.3 Gas Flow Calculations . . . 25
10.1.4 Tank Ullage Volume Dynamics . . . 26
10.1.5 Tank Gas Mass and Pressure . . . 26
10.2 Startup Behavior . . . 26
10.3 End State Behavior . . . 26

11 Complete DDP Algorithm **27**
11.1 Full Algorithm Pseudocode . . . 27
11.2 Forward Rollout Details . . . 27
11.3 Backward Pass Details . . . 28
11.4 Forward Line Search Details . . . 29

12 Cost Function Detailed Derivation **29**
12.1 Thrust Tracking Cost . . . 29
12.2 Mixture Ratio Tracking Cost . . . 30
12.3 Gas Consumption Cost . . . 30
12.4 Constraint Penalties . . . 30

13 Linearization via Finite Differences **30**
13.1 State Jacobian $\mathbf{A}_k$ . . . 30
13.2 Control Jacobian $\mathbf{B}_k$ . . . 30
13.3 Cost Derivatives . . . 31
13.3.1 State Gradient $\ell_{\mathbf{x}}$ . . . 31
13.3.2 Control Gradient $\ell_{\mathbf{u}}$ . . . 31
13.3.3 State Hessian $\ell_{\mathbf{xx}}$ . . . 31
13.3.4 Control Hessian $\ell_{\mathbf{uu}}$ . . . 31
13.3.5 Cross-Derivative $\ell_{\mathbf{ux}}$ . . . 31

14 Numerical Stability **31**
14.1 Cholesky Decomposition for Positive Definiteness . . . 31
14.2 Condition Number Check . . . 32
14.3 Value Function Hessian Projection . . . 32

15 Constraint Handling **32**
15.1 Hard Constraints . . . 32
15.2 Constraint Violation Computation . . . 32

16 Implementation Parameters **33**
16.1 Default Configuration . . . 33
16.2 Physical Constants . . . 33

17 Summary **33**

1 Introduction

This document provides a comprehensive mathematical treatment of the robust Differential Dynamic Programming (DDP) controller for liquid rocket engine control. We derive all equations from first principles, beginning with fundamental physics and building up to the complete control framework.

1.1 Who Should Read This Document

This document is designed for:

- Engineers new to optimal control who want to understand DDP from the ground up
- Students learning control theory who need detailed derivations
- Practitioners implementing rocket engine controllers who need complete mathematical foundations
- Anyone who wants to understand every step of the controller logic

We assume only basic knowledge of:

- Calculus (derivatives, integrals)
- Linear algebra (matrices, vectors)
- Basic physics (ideal gas law, conservation laws)

All other concepts are explained from first principles.

1.2 Document Structure

The document is organized as follows:

1. **Background Theory**: Introduction to optimal control and DDP
2. **First Principles Physics**: Derivation of all physical laws from basics
3. **System Modeling**: Complete mathematical model of the rocket engine system
4. **Control Problem Formulation**: How we frame the control problem
5. **DDP Algorithm**: Step-by-step explanation of the DDP algorithm
6. **Controller Implementation**: Detailed walkthrough of the control loop
7. **Numerical Methods**: How we compute derivatives and solve equations

1.3 System Overview

The system consists of:

- **COPV** (Composite Overwrapped Pressure Vessel): High-pressure gas storage (typically 2750 psi, 4.5-6L volume)
- **Regulator**: Pressure-reducing valve maintaining fixed output pressure (typically 1000 psi)
- **Solenoid Valves**: PWM-controlled valves for fuel and oxidizer pressurization
- **Propellant Tanks**: Fuel (RP-1) and oxidizer (LOX) tanks with ullage volumes
- **Engine**: Combustion chamber producing thrust from propellant flow

1.4 Control Objective

The controller must:

1. Track reference thrust $F_{\text{ref}}(t)$ and mixture ratio $\text{MR}_{\text{ref}}(t)$
2. Minimize gas consumption from COPV
3. Satisfy constraints (pressure limits, mixture ratio bounds, etc.)
4. Handle uncertainties and disturbances robustly

2 Background: Optimal Control Theory

Before diving into the specific controller, we need to understand the general framework of optimal control. This section provides the theoretical foundation.

2.1 What is Optimal Control?

Optimal control is the problem of finding a control input $\mathbf{u}(t)$ that minimizes a cost function J while satisfying system dynamics and constraints.

Intuitive Example: Imagine driving a car. You want to:

- Minimize fuel consumption (cost)
- Follow the speed limit (constraints)
- Arrive at your destination (dynamics: position changes with velocity)

Optimal control finds the best way to press the gas pedal over time.

2.2 The General Optimal Control Problem

Given a system:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t), t) \tag{1}$$

where:

- $\mathbf{x}(t)$ is the **state** (e.g., position, velocity, pressure)
- $\mathbf{u}(t)$ is the **control** (e.g., throttle, valve opening)
- $\mathbf{f}$ is the **dynamics** (how state evolves)

We want to minimize:

$$J = \int_0^T \ell(\mathbf{x}(t), \mathbf{u}(t), t)dt + \phi(\mathbf{x}(T)) \tag{2}$$

where:

- ℓ is the **running cost** (cost at each moment)
- ϕ is the **terminal cost** (cost at the end)

2.3 Discrete-Time Formulation

In practice, we work with discrete time steps. Instead of continuous time t, we have time steps $k = 0, 1, 2, \ldots, N-1$.

The dynamics become:

$$\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, k) \tag{3}$$

The cost becomes:

$$J = \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k, k) + \phi(\mathbf{x}_N) \tag{4}$$

Why discrete? Real controllers run on computers with fixed time steps (e.g., 100 Hz = 0.01 s per step).

2.4 What is Differential Dynamic Programming (DDP)?

DDP is an algorithm for solving optimal control problems. It works by:

1. Starting with an initial guess for the control sequence
2. Linearizing the dynamics around the current trajectory
3. Computing how to improve the control (backward pass)
4. Testing the improved control (forward pass)
5. Repeating until convergence

Key Insight: DDP uses the structure of the problem (dynamics are known) to efficiently find optimal controls, unlike generic optimization that treats it as a black box.

2.5 The Bellman Principle of Optimality

The fundamental principle behind DDP is Bellman's principle of optimality:

"An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision."

In mathematical terms, if we define the **value function**:

$$V_k(\mathbf{x}_k) = \min_{\mathbf{u}_k, \mathbf{u}_{k+1}, \ldots} \left[\sum_{j=k}^{N-1} \ell(\mathbf{x}_j, \mathbf{u}_j, j) + \phi(\mathbf{x}_N) \right] \tag{5}$$

Then Bellman's equation says:

$$V_k(\mathbf{x}_k) = \min_{\mathbf{u}_k} \left[\ell(\mathbf{x}_k, \mathbf{u}_k, k) + V_{k+1}(\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)) \right] \tag{6}$$

Intuition: The best cost-to-go from state $\mathbf{x}_k$ is the minimum over all controls $\mathbf{u}_k$ of: (immediate cost) + (best cost-to-go from next state).

This is a **recursive** relationship: to know V_k, we need V_{k+1}. So we compute backwards from the end ($V_N = \phi$) to the beginning (V_0).

2.6 Why DDP Works

DDP is powerful because:

1. **Exploits structure**: Uses the known dynamics to compute gradients efficiently
2. **Second-order information**: Uses Hessians (curvature) for faster convergence
3. **Feedback gains**: Produces not just open-loop control, but feedback gains for robustness
4. **Handles constraints**: Can incorporate constraints through penalties or projections

3 First Principles Physics

3.1 Ideal Gas Law

The fundamental relationship for gas pressure is the ideal gas law. This law comes from kinetic theory of gases, which models gas as many tiny particles bouncing around.

3.1.1 Derivation from Kinetic Theory

Imagine a box of volume V containing N gas molecules, each with mass m_0. The molecules move randomly and collide with the walls.

When a molecule hits a wall, it exerts a force. The total force per unit area (pressure) is:

$$P = \frac{N m_0 \bar{v}^2}{3V} \tag{7}$$

where $\bar{v}^2$ is the average squared velocity.

From thermodynamics, we know that kinetic energy is related to temperature:

$$\frac{1}{2} m_0 \bar{v}^2 = \frac{3}{2} k_B T \tag{8}$$

where k_B is Boltzmann's constant.

Combining these and defining $R = k_B/m_0$ (specific gas constant) and $m = N m_0$ (total mass):

$$P = \frac{mRT}{V} \tag{9}$$

For real gases, we add a compressibility factor Z:

$$P = \frac{mRT}{VZ} \tag{10}$$

where:

- P = pressure [Pa] - force per unit area
- m = gas mass [kg] - total mass of gas
- R = specific gas constant [J/(kg·K)] = 296.8 for N_2 - depends on gas type
- T = temperature [K] - average kinetic energy of molecules
- V = volume [m^3] - space the gas occupies
- Z = compressibility factor (1.0 for ideal gas) - correction for real gas effects

Key Insight: Pressure increases if you:

- Add more gas (increase m)
- Heat the gas (increase T)
- Compress the gas (decrease V)

For our system: We use this to compute tank pressures from gas mass, temperature, and ullage volume.

3.2 Polytropic Process

For real systems, gas expansion/compression follows a polytropic process:

$$PV^n = \text{constant} \tag{11}$$

where n is the polytropic exponent:

- $n = 1.0$: Isothermal (constant temperature)
- $n = 1.4$: Adiabatic (no heat transfer, γ for diatomic gas)
- $n = 1.2$: Typical for blowdown (some heat transfer with tank walls)

The temperature evolution follows:

$$T = T_0 \left(\frac{\rho}{\rho_0}\right)^{n-1} = T_0 \left(\frac{m/V}{m_0/V_0}\right)^{n-1} \tag{12}$$

where $\rho = m/V$ is the gas density.

3.3 Compressible Gas Flow

For gas flow through orifices (valves, regulator), we use compressible flow equations.

3.3.1 Critical Pressure Ratio

For choked (sonic) flow, the critical pressure ratio is:

$$\left(\frac{P_{\text{down}}}{P_{\text{up}}}\right)_{\text{crit}} = \left(\frac{2}{\gamma+1}\right)^{\frac{\gamma}{\gamma-1}} \tag{13}$$

For N_2 with $\gamma = 1.4$:

$$\left(\frac{P_{\text{down}}}{P_{\text{up}}}\right)_{\text{crit}} = \left(\frac{2}{2.4}\right)^{\frac{1.4}{0.4}} = 0.528 \tag{14}$$

3.3.2 Choked Flow

When $P_{\text{down}}/P_{\text{up}} < 0.528$, flow is choked (sonic at orifice):

$$\dot{m} = C_d A P_{\text{up}} \sqrt{\frac{\gamma}{RT_{\text{up}}}} \left(\frac{2}{\gamma+1}\right)^{\frac{\gamma+1}{2(\gamma-1)}} \tag{15}$$

Defining the choked flow constant:

$$C^* = \sqrt{\gamma \left(\frac{2}{\gamma+1}\right)^{\frac{\gamma+1}{\gamma-1}}} \tag{16}$$

The mass flow rate becomes:

$$\dot{m} = C_d A \frac{P_{\text{up}}}{\sqrt{RT_{\text{up}}}} C^* \tag{17}$$

3.3.3 Subsonic Flow

When $P_{\text{down}}/P_{\text{up}} \geq 0.528$, flow is subsonic:

$$\dot{m} = C_d A P_{\text{up}} \sqrt{\frac{2\gamma}{(\gamma-1)RT_{\text{up}}}} \sqrt{\left(\frac{P_{\text{down}}}{P_{\text{up}}}\right)^{\frac{2}{\gamma}} - \left(\frac{P_{\text{down}}}{P_{\text{up}}}\right)^{\frac{\gamma+1}{\gamma}}} \tag{18}$$

3.4 Mass Conservation

For the COPV:

$$\dot{m}_{\text{copv}} = -\dot{m}_{\text{to_reg}} - \dot{m}_{\text{loss}} \tag{19}$$

For tank ullage:

$$\dot{m}_{\text{gas,F}} = \dot{m}_{\text{gas,F,in}} \quad \text{(when valve open)} \tag{20}$$

$$\dot{m}_{\text{gas,F}} = 0 \quad \text{(when valve closed, pure blowdown)} \tag{21}$$

3.5 Ullage Volume Dynamics

As propellant flows to the engine, ullage volume increases:

$$\dot{V}_{u,F} = \frac{\dot{m}_F}{\rho_F} \tag{22}$$

where:

- $\dot{V}_{u,F}$ = ullage volume growth rate [m^3/s]
- $\dot{m}_F$ = fuel mass flow rate to engine [kg/s]
- ρ_F = fuel density [kg/m^3] = 800 for RP-1

4 System Dynamics

4.1 State Vector

The complete state vector is:

$$\mathbf{x} = \begin{bmatrix} P_{\text{copv}} \\ P_{\text{reg}} \\ P_{u,F} \\ P_{u,O} \\ P_{d,F} \\ P_{d,O} \\ V_{u,F} \\ V_{u,O} \\ m_{\text{gas,copv}} \\ m_{\text{gas,F}} \\ m_{\text{gas,O}} \end{bmatrix} \in \mathbb{R}^{11} \tag{23}$$

where:

- P_{copv} = COPV pressure [Pa]
- P_{reg} = Regulator output pressure [Pa]
- $P_{u,F}$, $P_{u,O}$ = Upstream (tank) pressures [Pa]

- $P_{d,F}$, $P_{d,O}$ = Downstream (feed line) pressures [Pa]
- $V_{u,F}$, $V_{u,O}$ = Ullage volumes [m^3]
- $m_{\text{gas,copv}}$, $m_{\text{gas,F}}$, $m_{\text{gas,O}}$ = Gas masses [kg]

4.2 Control Vector

The control input is:

$$\mathbf{u} = \begin{bmatrix} u_F \\ u_O \end{bmatrix} \in [0,1]^2 \tag{24}$$

where u_F, u_O are duty cycles (0 = valve closed, 1 = valve fully open).

4.3 Discrete-Time Dynamics

The system evolves according to:

$$\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, \Delta t, \dot{m}_F, \dot{m}_O) \tag{25}$$

where $\dot{m}_F, \dot{m}_O$ are propellant mass flow rates (from engine physics).

4.3.1 COPV Gas Mass Update

$$m_{\text{gas,copv},k+1} = m_{\text{gas,copv},k} - \Delta t \left(\dot{m}_{\text{to_reg}} + \dot{m}_{\text{loss}}\right) \tag{26}$$

4.3.2 COPV Pressure

Using ideal gas law with polytropic temperature:

$$T_{\text{copv},k+1} = T_{\text{copv},0} \left(\frac{\rho_{\text{copv},k+1}}{\rho_{\text{copv},0}}\right)^{n-1} \tag{27}$$

$$P_{\text{copv},k+1} = \frac{m_{\text{gas,copv},k+1} R T_{\text{copv},k+1}}{V_{\text{copv}} Z} \tag{28}$$

4.3.3 Regulator Pressure

Fixed setpoint regulator:

$$P_{\text{reg},k+1} = \begin{cases} P_{\text{setpoint}} & \text{if } P_{\text{copv},k} \geq P_{\text{setpoint}} \\ P_{\text{copv},k} & \text{if } P_{\text{copv},k} < P_{\text{setpoint}} \end{cases} \tag{29}$$

With oscillations due to PWM control:

$$P_{\text{reg},k+1} = P_{\text{reg},k+1,\text{base}} + A_{\text{osc}} \sin(\phi_k) \tag{30}$$

where $A_{\text{osc}} = 0.06 P_{\text{setpoint}} \cdot u_{\text{total}}$ and ϕ_k is the PWM phase.

4.3.4 Gas Flow from COPV to Regulator

Using choked/subsonic flow equations:

$$\dot{m}_{\text{to_reg}} = \begin{cases} C_d A_{\text{reg}} \frac{P_{\text{copv}}}{\sqrt{RT_{\text{copv}}}} C^* & \text{if choked} \\ C_d A_{\text{reg}} P_{\text{copv}} \sqrt{\frac{2\gamma}{(\gamma-1)RT_{\text{copv}}}} \sqrt{\text{pr}^{\frac{2}{\gamma}} - \text{pr}^{\frac{\gamma+1}{\gamma}}} & \text{if subsonic} \end{cases} \tag{31}$$

where $\text{pr} = P_{\text{reg}}/P_{\text{copv}}$.
Flow is gated by control:

$$\dot{m}_{\text{to_reg}} = \dot{m}_{\text{to_reg,max}} \cdot u_{\text{total}} \tag{32}$$

where $u_{\text{total}} = \max(u_F, u_O)$.

4.3.5 Gas Flow from Regulator to Tanks

For fuel tank:

$$\dot{m}_{\text{gas,F}} = \begin{cases} C_d A_{\text{valve,F}} \frac{P_{\text{reg}}}{\sqrt{RT_{\text{reg}}}} C^* \cdot u_F & \text{if choked} \\ C_d A_{\text{valve,F}} P_{\text{reg}} \sqrt{\frac{2\gamma}{(\gamma-1)RT_{\text{reg}}}} \sqrt{\text{pr}_F^{\frac{2}{\gamma}} - \text{pr}_F^{\frac{\gamma+1}{\gamma}}} \cdot u_F & \text{if subsonic} \end{cases} \tag{33}$$

where $\text{pr}_F = P_{u,F}/P_{\text{reg}}$.

4.3.6 Ullage Volume Update

$$V_{u,F,k+1} = V_{u,F,k} + \Delta t \frac{\dot{m}_{F,\text{pressure_dependent}}}{\rho_F} \tag{34}$$

where $\dot{m}_{F,\text{pressure_dependent}}$ accounts for pressure-dependent flow:

$$\dot{m}_{F,\text{pressure_dependent}} = \dot{m}_F \sqrt{\frac{P_{u,F}}{P_{\text{ref},F}}} \tag{35}$$

This creates exponential blowdown: as pressure decreases, mass flow decreases, making decay exponential.

4.3.7 Tank Gas Mass Update

$$m_{\text{gas,F},k+1} = m_{\text{gas,F},k} + \Delta t \cdot \dot{m}_{\text{gas,F}} \tag{36}$$

4.3.8 Tank Pressure

Using ideal gas law with polytropic temperature:

$$T_{\text{gas,F},k+1} = T_{\text{gas,F},0} \left(\frac{\rho_{F,k+1}}{\rho_{F,0}} \right)^{n-1} \tag{37}$$

$$P_{u,F,k+1} = \frac{m_{\text{gas,F},k+1} R T_{\text{gas,F},k+1}}{V_{u,F,k+1} Z} \tag{38}$$

4.3.9 Feed Line Pressure Dynamics

First-order lag:

$$P_{d,F,k+1} = P_{d,F,k} + \Delta t \frac{P_{u,F,k} - P_{d,F,k}}{\tau_{\text{line},F}} \tag{39}$$

5 Differential Dynamic Programming

5.1 Problem Formulation

We seek to minimize:

$$J = \sum_{k=0}^{N-1} \ell(\mathbf{x}_k, \mathbf{u}_k, k) + \phi(\mathbf{x}_N) \tag{40}$$

subject to:

$$\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, \Delta t, \dot{m}_F, \dot{m}_O) \tag{41}$$

5.2 Running Cost

$$\ell(\mathbf{x}_k, \mathbf{u}_k, k) = q_F(F_k - F_{\text{ref},k})^2 + q_{\text{MR}}(\text{MR}_k - \text{MR}_{\text{ref},k})^2 + q_{\text{gas}}\Delta P_{\text{copv}} + q_{\text{switch}}\|\mathbf{u}_k - \mathbf{u}_{k-1}\|^2 + \text{constraint penalties} \tag{42}$$

5.2.1 Thrust Tracking

Normalized error to prevent blowup:

$$F_{\text{error}} = \frac{F_k - F_{\text{ref},k}}{\max(|F_{\text{ref},k}|, 1.0)} \tag{43}$$

$$\text{Cost}_F = \begin{cases} q_F F_{\text{error}}^2 \cdot 2.0 & \text{if } F_{\text{error}} < 0 \text{ (thrust too low)} \\ q_F F_{\text{error}}^2 & \text{if } F_{\text{error}} \geq 0 \end{cases} \tag{44}$$

5.2.2 Mixture Ratio Tracking

$$\text{MR}_{\text{error}} = \frac{\text{MR}_k - \text{MR}_{\text{ref},k}}{\max(|\text{MR}_{\text{ref},k}|, 1.0)} \tag{45}$$

$$\text{Cost}_{\text{MR}} = q_{\text{MR}}\text{MR}_{\text{error}}^2 \tag{46}$$

5.2.3 Gas Consumption

$$\Delta P_{\text{copv}} = \Delta t\,(c_F u_F + c_O u_O + \text{loss}) \tag{47}$$

$$\text{Cost}_{\text{gas}} = q_{\text{gas}}\Delta P_{\text{copv}} \tag{48}$$

5.3 Bellman Equation

The value function satisfies:

$$V_k(\mathbf{x}_k) = \min_{\mathbf{u}_k} \left[\ell(\mathbf{x}_k, \mathbf{u}_k, k) + V_{k+1}(\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k))\right] \tag{49}$$

5.4 Linearization

We linearize the dynamics around a nominal trajectory:

$$\delta \mathbf{x}_{k+1} = \mathbf{A}_k \delta \mathbf{x}_k + \mathbf{B}_k \delta \mathbf{u}_k \tag{50}$$

where:

$$\mathbf{A}_k = \frac{\partial \mathbf{f}}{\partial \mathbf{x}}\Big|_{\mathbf{x}_k, \mathbf{u}_k}, \quad \mathbf{B}_k = \frac{\partial \mathbf{f}}{\partial \mathbf{u}}\Big|_{\mathbf{x}_k, \mathbf{u}_k} \tag{51}$$

Computed via finite differences:

$$[\mathbf{A}_k]_{i,j} = \frac{f_i(\mathbf{x}_k + \epsilon \mathbf{e}_j, \mathbf{u}_k) - f_i(\mathbf{x}_k, \mathbf{u}_k)}{\epsilon} \tag{52}$$

5.5 Backward Pass

5.5.1 Cost Derivatives

$$\ell_{\mathbf{x}} = \frac{\partial \ell}{\partial \mathbf{x}}, \quad \ell_{\mathbf{u}} = \frac{\partial \ell}{\partial \mathbf{u}} \tag{53}$$

$$\ell_{\mathbf{xx}} = \frac{\partial^2 \ell}{\partial \mathbf{x}^2}, \quad \ell_{\mathbf{uu}} = \frac{\partial^2 \ell}{\partial \mathbf{u}^2}, \quad \ell_{\mathbf{ux}} = \frac{\partial^2 \ell}{\partial \mathbf{u} \partial \mathbf{x}} \tag{54}$$

Computed via finite differences with engine re-estimation when tank pressures change.

5.5.2 Q-Function

$$Q(\delta \mathbf{x}_k, \delta \mathbf{u}_k) = \ell(\mathbf{x}_k + \delta \mathbf{x}_k, \mathbf{u}_k + \delta \mathbf{u}_k) + V_{k+1}(\mathbf{f}(\mathbf{x}_k + \delta \mathbf{x}_k, \mathbf{u}_k + \delta \mathbf{u}_k)) \tag{55}$$

Expanding to second order:

$$Q \approx Q_0 + \mathbf{Q}_{\mathbf{x}}^T \delta \mathbf{x}_k + \mathbf{Q}_{\mathbf{u}}^T \delta \mathbf{u}_k + \frac{1}{2} \delta \mathbf{x}_k^T \mathbf{Q}_{\mathbf{xx}} \delta \mathbf{x}_k + \frac{1}{2} \delta \mathbf{u}_k^T \mathbf{Q}_{\mathbf{uu}} \delta \mathbf{u}_k + \delta \mathbf{u}_k^T \mathbf{Q}_{\mathbf{ux}} \delta \mathbf{x}_k \tag{56}$$

where:

$$\mathbf{Q}_{\mathbf{x}} = \ell_{\mathbf{x}} + \mathbf{A}_k^T \mathbf{V}_{\mathbf{x},k+1} \tag{57}$$

$$\mathbf{Q}_{\mathbf{u}} = \ell_{\mathbf{u}} + \mathbf{B}_k^T \mathbf{V}_{\mathbf{x},k+1} \tag{58}$$

$$\mathbf{Q}_{\mathbf{xx}} = \ell_{\mathbf{xx}} + \mathbf{A}_k^T \mathbf{V}_{\mathbf{xx},k+1} \mathbf{A}_k \tag{59}$$

$$\mathbf{Q}_{\mathbf{uu}} = \ell_{\mathbf{uu}} + \mathbf{B}_k^T \mathbf{V}_{\mathbf{xx},k+1} \mathbf{B}_k \tag{60}$$

$$\mathbf{Q}_{\mathbf{ux}} = \ell_{\mathbf{ux}} + \mathbf{B}_k^T \mathbf{V}_{\mathbf{xx},k+1} \mathbf{A}_k \tag{61}$$

5.5.3 Optimal Control Update

Minimizing Q with respect to $\delta \mathbf{u}_k$:

$$\frac{\partial Q}{\partial \delta \mathbf{u}_k} = \mathbf{Q}_{\mathbf{u}} + \mathbf{Q}_{\mathbf{uu}} \delta \mathbf{u}_k + \mathbf{Q}_{\mathbf{ux}} \delta \mathbf{x}_k = 0 \tag{62}$$

Solving:

$$\delta \mathbf{u}_k^* = -\mathbf{Q}_{\mathbf{uu}}^{-1} \mathbf{Q}_{\mathbf{u}} - \mathbf{Q}_{\mathbf{uu}}^{-1} \mathbf{Q}_{\mathbf{ux}} \delta \mathbf{x}_k \tag{63}$$

Defining feedforward and feedback gains:

$$\mathbf{k}_k = -\mathbf{Q}_{\mathbf{uu}}^{-1} \mathbf{Q}_{\mathbf{u}}, \quad \mathbf{K}_k = -\mathbf{Q}_{\mathbf{uu}}^{-1} \mathbf{Q}_{\mathbf{ux}} \tag{64}$$

5.5.4 Regularization

To ensure $\mathbf{Q_{uu}}$ is positive definite:

$$\mathbf{Q}_{\mathbf{uu},\text{reg}} = \mathbf{Q_{uu}} + \lambda \mathbf{I} \tag{65}$$

where λ is Levenberg-Marquardt regularization.

5.5.5 Value Function Update

$$\mathbf{V}_{\mathbf{x},k} = \mathbf{Q_x} + \mathbf{K}_k^T \mathbf{Q_{uu}} \mathbf{k}_k + \mathbf{K}_k^T \mathbf{Q_u} + \mathbf{Q}_{\mathbf{ux}}^T \mathbf{k}_k \tag{66}$$

$$\mathbf{V}_{\mathbf{xx},k} = \mathbf{Q_{xx}} + \mathbf{K}_k^T \mathbf{Q_{uu}} \mathbf{K}_k + \mathbf{K}_k^T \mathbf{Q_{ux}} + \mathbf{Q}_{\mathbf{ux}}^T \mathbf{K}_k \tag{67}$$

5.6 Forward Pass

5.6.1 Line Search

We update the control sequence:

$$\mathbf{u}_k^{\text{new}} = \mathbf{u}_k + \alpha \mathbf{k}_k + \mathbf{K}_k (\mathbf{x}_k^{\text{new}} - \mathbf{x}_k) \tag{68}$$

where α is found via line search to minimize cost.

5.6.2 Forward Rollout

$$\mathbf{x}_{k+1}^{\text{new}} = \mathbf{f}(\mathbf{x}_k^{\text{new}}, \mathbf{u}_k^{\text{new}}, \Delta t, \dot{m}_F, \dot{m}_O) \tag{69}$$

6 Robustification

6.1 Uncertainty Inflation

We inflate the value function Hessian to account for uncertainties:

$$\mathbf{V}_{\mathbf{xx},k} \leftarrow \mathbf{V}_{\mathbf{xx},k} + \gamma \text{diag}(\bar{\mathbf{w}}^2) \tag{70}$$

where $\bar{\mathbf{w}}$ are residual bounds and γ is the robustification weight.

7 Constraints

7.1 Soft Constraints

Constraint violations are penalized in the cost:

$$\text{Cost}_{\text{constraint}} = w_{\text{constraint}} \max(0, \text{violation})^2 \tag{71}$$

7.2 Hard Constraints

Hard constraints are checked and violations reported:

- COPV minimum pressure
- Ullage maximum volume
- Mixture ratio bounds
- Injector pressure drop minimum

8 Numerical Implementation

This section explains how we actually compute the derivatives and solve the equations numerically. These are the "nuts and bolts" of the implementation.

8.1 Why Finite Differences?

We need to compute derivatives like $\frac{\partial \mathbf{f}}{\partial \mathbf{x}}$ and $\frac{\partial \ell}{\partial \mathbf{x}}$.

Option 1: Analytical derivatives - Derive formulas by hand. This is:

- **Pros**: Exact, fast
- **Cons**: Very tedious for complex functions, error-prone

Option 2: Automatic differentiation - Use libraries like JAX/TensorFlow. This is:

- **Pros**: Exact, automatic
- **Cons**: Requires rewriting code, adds dependencies

Option 3: Finite differences - Approximate derivatives numerically. This is:

- **Pros**: Simple, works with any function, no code changes needed
- **Cons**: Approximate (but very accurate with small ϵ)

We use finite differences because it's simple and accurate enough.

8.2 Finite Differences: The Basic Idea

The derivative is defined as:

$$\frac{\partial f}{\partial x_i} = \lim_{\epsilon \to 0} \frac{f(\mathbf{x} + \epsilon \mathbf{e}_i) - f(\mathbf{x})}{\epsilon} \tag{72}$$

We approximate this by using a small but finite ϵ:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(\mathbf{x} + \epsilon \mathbf{e}_i) - f(\mathbf{x})}{\epsilon} \tag{73}$$

where $\mathbf{e}_i$ is the i-th unit vector (zeros except 1 at position i).

Example: For $f(x, y) = x^2 + y^2$:

$$\frac{\partial f}{\partial x} \approx \frac{f(x + \epsilon, y) - f(x, y)}{\epsilon} = \frac{(x + \epsilon)^2 + y^2 - (x^2 + y^2)}{\epsilon} = \frac{2x\epsilon + \epsilon^2}{\epsilon} = 2x + \epsilon \tag{74}$$

With $\epsilon = 10^{-6}$, this is extremely close to the true derivative $2x$.

8.3 Choosing ϵ

The choice of ϵ is critical:

- **Too small** (e.g., 10^{-12}): Numerical roundoff errors dominate
- **Too large** (e.g., 10^{-3}): Approximation error is large
- **Just right** (e.g., 10^{-6}): Good balance

We use $\epsilon = 10^{-6}$ as a standard choice.

8.4 Computing the Dynamics Jacobian $\mathbf{A}_k$

The dynamics Jacobian $\mathbf{A}_k = \frac{\partial \mathbf{f}}{\partial \mathbf{x}}$ is an 11×11 matrix.

Algorithm:

```
A = zeros(11, 11)
for i = 0 to 10:
    x_pert = x.copy()
    x_pert[i] += epsilon
    f_pert = f(x_pert, u, dt, mdot_F, mdot_O)
    f_nom = f(x, u, dt, mdot_F, mdot_O)
    A[:, i] = (f_pert - f_nom) / epsilon
```

What this does: For each state component i, we:

1. Perturb it by ϵ
2. Simulate dynamics forward
3. See how all 11 states change
4. This gives column i of $\mathbf{A}_k$

Example: If we increase P_{copv} by ϵ, how does this affect all 11 states?

- P_{copv} itself increases (obviously)
- P_{reg} might increase (if regulator can supply more)
- Gas flow rates increase (higher upstream pressure)
- Tank pressures might increase (more gas flowing in)
- etc.

The i-th column of $\mathbf{A}_k$ captures all these effects.

8.5 Computing the Control Jacobian $\mathbf{B}_k$

Similarly, $\mathbf{B}_k = \frac{\partial \mathbf{f}}{\partial \mathbf{u}}$ is an 11×2 matrix.

Algorithm:

```
B = zeros(11, 2)
for j = 0 to 1:
    u_pert = u.copy()
    u_pert[j] = clip(u_pert[j] + epsilon, 0, 1)
    f_pert = f(x, u_pert, dt, mdot_F, mdot_O)
    f_nom = f(x, u, dt, mdot_F, mdot_O)
    B[:, j] = (f_pert - f_nom) / epsilon
```

What this does: For each control component j (fuel or oxidizer valve):

1. Increase duty cycle by ϵ
2. Simulate dynamics forward
3. See how all 11 states change

Example: If we increase u_F (fuel valve) by ϵ:

- More gas flows to fuel tank ($m_{\text{gas,F}}$ increases)
- Fuel tank pressure increases ($P_{u,F}$ increases)
- Fuel mass flow to engine increases ($\dot{m}_F$ increases)
- Ullage volume grows faster ($V_{u,F}$ increases faster)
- etc.

8.6 Computing Cost Derivatives

The cost function $\ell(\mathbf{x}, \mathbf{u})$ depends on:

- State $\mathbf{x}$ (pressures, volumes, gas masses)
- Control $\mathbf{u}$ (duty cycles)
- Engine estimates (thrust, mixture ratio, mass flows)

State gradient $\ell_{\mathbf{x}}$:

```
lx = zeros(11)
for i = 0 to 10:
    x_pert = x.copy()
    x_pert[i] += epsilon

    # Re-estimate engine if tank pressures changed
    if i in [IDX_P_U_F, IDX_P_U_O]:
        eng_est_pert = engine_wrapper.estimate_from_pressures(
            x_pert[IDX_P_U_F], x_pert[IDX_P_U_O]
        )
    else:
        eng_est_pert = eng_est

    cost_pert = running_cost(x_pert, u, eng_est_pert, F_ref, MR_ref, ...)
    cost_nom = running_cost(x, u, eng_est, F_ref, MR_ref, ...)
    lx[i] = (cost_pert - cost_nom) / epsilon
```

Control gradient $\ell_{\mathbf{u}}$:

```
lu = zeros(2)
for j = 0 to 1:
    u_pert = u.copy()
    u_pert[j] = clip(u_pert[j] + epsilon, 0, 1)
    cost_pert = running_cost(x, u_pert, eng_est, F_ref, MR_ref, ...)
    cost_nom = running_cost(x, u, eng_est, F_ref, MR_ref, ...)
    lu[j] = (cost_pert - cost_nom) / epsilon
```

Why re-estimate engine? When tank pressures change, engine performance changes (thrust, mixture ratio, mass flows). We need accurate derivatives, so we re-estimate.

8.7 Computing Hessians

For second derivatives, we use the central difference formula:

$$\frac{\partial^2 f}{\partial x_i^2} \approx \frac{f(\mathbf{x} + \epsilon \mathbf{e}_i) - 2f(\mathbf{x}) + f(\mathbf{x} - \epsilon \mathbf{e}_i)}{\epsilon^2} \tag{75}$$

Why central difference? More accurate than forward difference for second derivatives.

Diagonal approximation: We only compute diagonal elements of Hessians (off-diagonal set to zero). This is:

- Much faster ($O(n)$ instead of $O(n^2)$)
- Usually sufficient (cross-terms are often small)

8.8 Cholesky Decomposition for Positive Definiteness

In the backward pass, we need to invert $\mathbf{Q_{uu}}$:

$$\mathbf{k}_k = -\mathbf{Q_{uu}}^{-1}\mathbf{Q_u} \tag{76}$$

For this to work, $\mathbf{Q_{uu}}$ must be **positive definite** (all eigenvalues > 0).

Why? If $\mathbf{Q_{uu}}$ is not positive definite:

- It might not be invertible
- Even if invertible, the solution might be unstable
- The optimization problem might not have a minimum

How to check? Use Cholesky decomposition:

$$\mathbf{Q_{uu}} = \mathbf{L}\mathbf{L}^T \tag{77}$$

where $\mathbf{L}$ is lower triangular.

Key fact: A matrix is positive definite if and only if it has a Cholesky decomposition.

Algorithm:

```
lambda_reg = lambda_init
for attempt = 0 to max_attempts:
    Quu_reg = Quu + lambda_reg * I
    try:
        L = cholesky(Quu_reg)  # If succeeds, matrix is positive definite
        break
    except LinAlgError:
        lambda_reg *= 10  # Add more regularization
```

What is regularization? Adding $\lambda\mathbf{I}$ to the matrix:

- Makes eigenvalues larger (more positive)
- Ensures positive definiteness
- But: Too much regularization makes the controller conservative (less responsive)

Balance: We want the minimum regularization that ensures stability.

8.9 Convergence Criteria

The DDP algorithm iterates until convergence. We check:

$$|\text{Cost}_{\text{new}} - \text{Cost}_{\text{old}}| < \epsilon_{\text{conv}} \tag{78}$$

where $\epsilon_{\text{conv}} = 10^{-3}$ (typical value).

What does this mean? If the cost changes by less than 0.001 between iterations, we've converged.

Why not 10^{-6}? Too strict - might never converge due to numerical errors.

Why not 10^{-1}? Too loose - might stop before finding good solution.

Other stopping criteria:

- Maximum iterations reached (e.g., 20 iterations)
- Regularization too high (e.g., $\lambda > 10^6$) - indicates numerical issues
- Cost increases significantly - indicates divergence

8.10 Value Function Hessian Projection

The value function Hessian $\mathbf{V_{xx}}$ should be **positive semi-definite** (all eigenvalues ≥ 0).

Why? The value function represents "cost-to-go" - it should always be non-negative, and the Hessian captures curvature.

Problem: Due to numerical errors, $\mathbf{V_{xx}}$ might have negative eigenvalues.

Solution: Project to positive semi-definite using eigendecomposition:

1. Compute eigendecomposition:
$$\mathbf{V_{xx}} = \mathbf{Q\Lambda Q}^T \tag{79}$$
where $\mathbf{\Lambda}$ is diagonal (eigenvalues), $\mathbf{Q}$ is orthogonal (eigenvectors).

2. Project negative eigenvalues to zero:
$$\mathbf{\Lambda}_{\text{proj}} = \max(\mathbf{\Lambda}, 0) \tag{80}$$

3. Reconstruct:
$$\mathbf{V}_{\mathbf{xx},\text{proj}} = \mathbf{Q\Lambda}_{\text{proj}}\mathbf{Q}^T \tag{81}$$

Intuition: We're "flattening out" any negative curvature, which would indicate the value function decreases in some direction (impossible for a cost function).

9 Complete Controller Logic Walkthrough

This section provides a step-by-step walkthrough of the controller logic, as if you were debugging it line-by-line. We'll trace through exactly what happens when the controller runs.

9.1 High-Level Control Loop

The controller runs in a loop at a fixed frequency (typically 100 Hz = every 0.01 seconds). Each iteration:

1. **Read sensors**: Get current pressures, temperatures, etc.
2. **Build state**: Convert sensor readings into state vector

3. **Generate reference**: Compute desired thrust and mixture ratio
4. **Solve DDP**: Find optimal control sequence
5. **Apply control**: Send commands to valves
6. **Update internal state**: Store information for next iteration

Let's go through each step in detail.

9.2 Step 1: Read Sensors

The controller receives a **Measurement** object containing:

- P_{copv}: COPV pressure [Pa]
- P_{reg}: Regulator output pressure [Pa]
- $P_{u,F}$, $P_{u,O}$: Tank (upstream) pressures [Pa]
- $P_{d,F}$, $P_{d,O}$: Feed line (downstream) pressures [Pa]
- Timestamp

These come from pressure transducers in the physical system.
Example: At time $t = 0.5$ s:

- $P_{\text{copv}} = 18.96 \times 10^6$ Pa (2750 psi)
- $P_{\text{reg}} = 6.89 \times 10^6$ Pa (1000 psi)
- $P_{u,F} = 5.0 \times 10^6$ Pa (725 psi)
- $P_{u,O} = 5.0 \times 10^6$ Pa (725 psi)

9.3 Step 2: Build State Vector

The controller maintains an internal state vector $\mathbf{x} \in \mathbb{R}^{11}$:

$$\mathbf{x} = \begin{bmatrix} P_{\text{copv}} \\ P_{\text{reg}} \\ P_{u,F} \\ P_{u,O} \\ P_{d,F} \\ P_{d,O} \\ V_{u,F} \\ V_{u,O} \\ m_{\text{gas,copv}} \\ m_{\text{gas,F}} \\ m_{\text{gas,O}} \end{bmatrix} \tag{82}$$

Direct measurements (from sensors):

- P_{copv}, P_{reg}, $P_{u,F}$, $P_{u,O}$, $P_{d,F}$, $P_{d,O}$ come directly from sensors

Estimated/internal states:

- $V_{u,F}$, $V_{u,O}$: Ullage volumes - tracked internally, initialized from config

- $m_{\text{gas,copv}}$, $m_{\text{gas,F}}$, $m_{\text{gas,O}}$: Gas masses - computed from pressure using ideal gas law

How gas masses are computed: From ideal gas law: $P = mRT/(VZ)$, so:

$$m = \frac{PVZ}{RT} \tag{83}$$

For COPV:

$$m_{\text{gas,copv}} = \frac{P_{\text{copv}} V_{\text{copv}} Z}{RT_{\text{copv}}} \tag{84}$$

For tanks:

$$m_{\text{gas,F}} = \frac{P_{u,F} V_{u,F} Z}{RT_{\text{gas,F}}} \tag{85}$$

Initialization: On first step, gas masses are computed from initial pressures and volumes. On subsequent steps, they're updated by the dynamics model.

9.4 Step 3: Generate Reference

The controller needs to know what thrust F_{ref} and mixture ratio MR_{ref} to track.

Reference generation logic:

1. If command is "thrust desired": $F_{\text{ref}} = F_{\text{desired}}$
2. If command is "altitude goal": Compute F_{ref} needed to achieve altitude (using vehicle dynamics)
3. MR_{ref} is typically constant (e.g., 2.0) or from Layer 2 optimization

The reference is generated for the entire prediction horizon (N steps ahead).

Example: Command says "maintain 1000 N thrust for 0.5 seconds":

$$F_{\text{ref}} = [1000, 1000, 1000, \ldots, 1000] \quad \text{(50 values for } N = 50) \tag{86}$$

9.5 Step 4: Solve DDP

This is the core of the controller. The DDP solver finds the optimal control sequence.

9.5.1 4.1: Initialize Control Sequence

Start with an initial guess $\mathbf{u}_{\text{init}}$. Options:

- **Warm start**: Use previous solution (shifted by one step)
- **Cold start**: Use constant values (e.g., 0.1 for both valves)
- **Heuristic**: Use simple rule (e.g., proportional to thrust error)

Example: Warm start from previous iteration:

$$\mathbf{u}_{\text{init}}[k] = \begin{cases} \mathbf{u}_{\text{prev}}[k+1] & \text{if } k < N-1 \\ \mathbf{u}_{\text{prev}}[N-1] & \text{if } k = N-1 \end{cases} \tag{87}$$

9.5.2 4.2: Forward Rollout

Simulate the system forward using the current control sequence:

For each time step $k = 0, 1, \ldots, N-1$:

1. Get current state: $\mathbf{x}_k$
2. Get current control: $\mathbf{u}_k$
3. Estimate engine performance:

$$\text{eng_est}_k = \text{engine_wrapper.estimate_from_pressures}(P_{u,F,k}, P_{u,O,k}) \tag{88}$$

   This returns: F_k, MR_k, $\dot{m}_{F,k}$, $\dot{m}_{O,k}$

4. Step dynamics:

$$\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, \Delta t, \dot{m}_{F,k}, \dot{m}_{O,k}) \tag{89}$$

5. Compute cost:

$$\ell_k = \text{running_cost}(\mathbf{x}_k, \mathbf{u}_k, \text{eng_est}_k, F_{\text{ref},k}, \text{MR}_{\text{ref},k}) \tag{90}$$

Total cost:

$$J = \sum_{k=0}^{N-1} \ell_k \tag{91}$$

9.5.3 4.3: Backward Pass

Compute how to improve the control. This is the heart of DDP.

Initialize terminal value function:

$$\mathbf{V}_{\mathbf{x},N} = \mathbf{0}, \quad \mathbf{V}_{\mathbf{xx},N} = \mathbf{0} \tag{92}$$

For each time step $k = N-1, N-2, \ldots, 0$ **(backwards!)**:

1. **Linearize dynamics**: Compute how small changes in state/control affect next state

$$\mathbf{A}_k = \frac{\partial \mathbf{f}}{\partial \mathbf{x}}\Big|_{\mathbf{x}_k, \mathbf{u}_k}, \quad \mathbf{B}_k = \frac{\partial \mathbf{f}}{\partial \mathbf{u}}\Big|_{\mathbf{x}_k, \mathbf{u}_k} \tag{93}$$

   Computed via finite differences (see Section on Numerical Methods).

2. **Compute cost derivatives**: How does cost change with state/control?

$$\ell_{\mathbf{x}}, \ell_{\mathbf{u}}, \ell_{\mathbf{xx}}, \ell_{\mathbf{uu}}, \ell_{\mathbf{ux}} = \text{cost_derivatives}(\mathbf{x}_k, \mathbf{u}_k, \ldots) \tag{94}$$

3. **Compute Q-function derivatives**: Combine cost and value function

$$\mathbf{Q}_{\mathbf{x}} = \ell_{\mathbf{x}} + \mathbf{A}_k^T \mathbf{V}_{\mathbf{x},k+1} \tag{95}$$
$$\mathbf{Q}_{\mathbf{u}} = \ell_{\mathbf{u}} + \mathbf{B}_k^T \mathbf{V}_{\mathbf{x},k+1} \tag{96}$$
$$\mathbf{Q}_{\mathbf{xx}} = \ell_{\mathbf{xx}} + \mathbf{A}_k^T \mathbf{V}_{\mathbf{xx},k+1} \mathbf{A}_k \tag{97}$$
$$\mathbf{Q}_{\mathbf{uu}} = \ell_{\mathbf{uu}} + \mathbf{B}_k^T \mathbf{V}_{\mathbf{xx},k+1} \mathbf{B}_k \tag{98}$$
$$\mathbf{Q}_{\mathbf{ux}} = \ell_{\mathbf{ux}} + \mathbf{B}_k^T \mathbf{V}_{\mathbf{xx},k+1} \mathbf{A}_k \tag{99}$$

   Intuition: $\mathbf{Q}_{\mathbf{u}}$ tells us "if I increase control by $\delta\mathbf{u}$, how much does total cost change?" We want to move in the direction that decreases cost.

4. **Regularize**: Ensure $\mathbf{Q_{uu}}$ is positive definite

$$\mathbf{Q_{uu,reg}} = \mathbf{Q_{uu}} + \lambda \mathbf{I} \tag{100}$$

5. **Compute gains**: How should we change control?

$$\mathbf{k}_k = -(\mathbf{Q_{uu,reg}})^{-1}\mathbf{Q_u}, \quad \mathbf{K}_k = -(\mathbf{Q_{uu,reg}})^{-1}\mathbf{Q_{ux}} \tag{101}$$

   Intuition:

   - $\mathbf{k}_k$: Feedforward gain - "change control by this amount regardless of state"
   - $\mathbf{K}_k$: Feedback gain - "if state deviates by $\delta\mathbf{x}$, change control by $\mathbf{K}_k\delta\mathbf{x}$"

6. **Update value function**: Propagate backwards

$$\mathbf{V}_{\mathbf{x},k} = \mathbf{Q_x} + \mathbf{K}_k^T\mathbf{Q_{uu,reg}}\mathbf{k}_k + \mathbf{K}_k^T\mathbf{Q_u} + \mathbf{Q_{ux}}^T\mathbf{k}_k \tag{102}$$

$$\mathbf{V}_{\mathbf{xx},k} = \mathbf{Q_{xx}} + \mathbf{K}_k^T\mathbf{Q_{uu,reg}}\mathbf{K}_k + \mathbf{K}_k^T\mathbf{Q_{ux}} + \mathbf{Q_{ux}}^T\mathbf{K}_k \tag{103}$$

9.5.4 4.4: Forward Line Search

Test the improved control and find the best step size.

Control update:

$$\mathbf{u}_k^{\text{new}} = \mathbf{u}_k + \alpha\mathbf{k}_k + \mathbf{K}_k(\mathbf{x}_k^{\text{new}} - \mathbf{x}_k) \tag{104}$$

where α is the step size (found via line search).

Line search algorithm:

1. Start with $\alpha = \alpha_{\text{init}}$ (e.g., 2.0)
2. For each α:
   (a) Compute new control sequence
   (b) Forward rollout with new control
   (c) Compute new cost J_{new}
   (d) If $J_{\text{new}} < J_{\text{old}}$: accept and break
3. If no improvement: reduce α (e.g., $\alpha \leftarrow \alpha/2$) and try again
4. Stop when $\alpha < \alpha_{\min}$ or improvement found

9.5.5 4.5: Iterate

Repeat steps 4.2-4.4 until:

- Cost converges: $|J_{\text{new}} - J_{\text{old}}| < \epsilon_{\text{conv}}$
- Maximum iterations reached
- Numerical issues (regularization too high)

9.6 Step 5: Apply Control

Take the first control from the optimal sequence:

$$\mathbf{u}_{\text{apply}} = \mathbf{u}_0^{\text{optimal}} \tag{105}$$

This is the **receding horizon** approach: solve for N steps ahead, but only apply the first step.

Why? At the next iteration, we'll have new sensor readings and solve again. This makes the controller robust to disturbances.

9.7 Step 6: Update Internal State

Store information for next iteration:

- Previous state: $\mathbf{x}_{\text{prev}} = \mathbf{x}$
- Previous applied control: $\mathbf{u}_{\text{prev}} = \mathbf{u}_{\text{apply}}$
- Previous DDP solution: $\mathbf{u}_{\text{seq,prev}} = \mathbf{u}^{\text{optimal}}$ (for warm start)
- Updated ullage volumes: $V_{u,F}$, $V_{u,O}$ (from dynamics)

9.8 Complete Example: One Control Step

Let's trace through a complete example with numbers.

Initial conditions:

- $P_{\text{copv}} = 18.96 \times 10^6$ Pa
- $P_{\text{reg}} = 6.89 \times 10^6$ Pa
- $P_{u,F} = 5.0 \times 10^6$ Pa
- $V_{u,F} = 0.01$ m^3 (10 L)
- Command: $F_{\text{ref}} = 1000$ N

Step 1: Read sensors (already have pressures above)
Step 2: Build state

$$\mathbf{x} = [18.96 \times 10^6, 6.89 \times 10^6, 5.0 \times 10^6, 5.0 \times 10^6, \ldots, 0.01, 0.01, \ldots]^T \tag{106}$$

Compute gas masses:

$$m_{\text{gas,copv}} = \frac{18.96 \times 10^6 \times 0.006 \times 1.0}{296.8 \times 293} = 0.130 \text{ kg} \tag{107}$$

Step 3: Generate reference

$$F_{\text{ref}} = [1000, 1000, \ldots, 1000] \quad \text{(50 values)} \tag{108}$$

Step 4: Solve DDP

1. Forward rollout: Simulate with current control, get $J = 5000$
2. Backward pass: Compute gains $\mathbf{k}_k$, $\mathbf{K}_k$
3. Line search: Find $\alpha = 1.5$ gives $J = 3000$ (improvement!)
4. Iterate: After 5 iterations, $J = 500$ (converged)

Step 5: Apply control

$$\mathbf{u}_{\text{apply}} = [0.3, 0.25] \quad \text{(30\% fuel valve, 25\% oxidizer valve)} \tag{109}$$

Step 6: Update state

- Store $\mathbf{x}_{\text{prev}}$, $\mathbf{u}_{\text{prev}}$
- Update $V_{u,F}$ based on propellant consumed

10 Detailed Dynamics Derivation

10.1 Complete State Update Equations

We now derive the complete discrete-time update equations for each state component.

10.1.1 COPV Gas Mass and Pressure

Starting from mass conservation:

$$m_{\text{gas,copv},k+1} = m_{\text{gas,copv},k} - \Delta t \left(\dot{m}_{\text{to_reg},k} + \dot{m}_{\text{loss}}\right) \tag{110}$$

The temperature update (polytropic):

$$T_{\text{copv},k+1} = T_{\text{copv},0} \left(\frac{m_{\text{gas,copv},k+1}/V_{\text{copv}}}{m_{\text{gas,copv},0}/V_{\text{copv}}}\right)^{n-1} = T_{\text{copv},0} \left(\frac{m_{\text{gas,copv},k+1}}{m_{\text{gas,copv},0}}\right)^{n-1} \tag{111}$$

Pressure from ideal gas law:

$$P_{\text{copv},k+1} = \frac{m_{\text{gas,copv},k+1} R T_{\text{copv},k+1}}{V_{\text{copv}} Z} \tag{112}$$

10.1.2 Regulator Pressure with Oscillations

The base regulator pressure:

$$P_{\text{reg,base},k+1} = \begin{cases} P_{\text{setpoint}} & \text{if } P_{\text{copv},k} \geq P_{\text{setpoint}} \\ P_{\text{copv},k} & \text{if } P_{\text{copv},k} < P_{\text{setpoint}} \end{cases} \tag{113}$$

With PWM oscillations (hardware-matching):

$$P_{\text{reg},k+1} = P_{\text{reg,base},k+1} + 0.06 P_{\text{setpoint}} \cdot u_{\text{total}} \cdot \sin(2\pi(u_F + u_O)) \tag{114}$$

Clamped to $\pm 10\%$ of setpoint.

10.1.3 Gas Flow Calculations

For COPV to regulator, we compute the pressure ratio:

$$\text{pr} = \frac{P_{\text{reg}}}{P_{\text{copv}}} \tag{115}$$

Critical ratio for N_2 ($\gamma = 1.4$):

$$\text{pr}_{\text{crit}} = \left(\frac{2}{2.4}\right)^{\frac{1.4}{0.4}} = 0.528 \tag{116}$$

If $\text{pr} < 0.528$ (choked):

$$C^* = \sqrt{1.4 \left(\frac{2}{2.4}\right)^{\frac{2.4}{0.4}}} = \sqrt{1.4 \cdot 0.528^6} \approx 0.684 \tag{117}$$

$$\dot{m}_{\text{to_reg,max}} = 0.7 \cdot 2 \times 10^{-5} \cdot \frac{P_{\text{copv}}}{\sqrt{296.8 \cdot T_{\text{copv}}}} \cdot 0.684 \tag{118}$$

If $\text{pr} \geq 0.528$ (subsonic):

$$\dot{m}_{\text{to_reg,max}} = 0.7 \cdot 2 \times 10^{-5} \cdot P_{\text{copv}} \sqrt{\frac{2.8}{0.4 \cdot 296.8 \cdot T_{\text{copv}}}} \sqrt{\text{pr}^{1.429} - \text{pr}^{1.714}} \tag{119}$$

Gated by control:

$$\dot{m}_{\text{to_reg}} = \dot{m}_{\text{to_reg,max}} \cdot \max(u_F, u_O) \tag{120}$$

10.1.4 Tank Ullage Volume Dynamics

The ullage volume grows as propellant is consumed:

$$V_{u,F,k+1} = V_{u,F,k} + \Delta t \frac{\dot{m}_{F,\text{pressure_dependent}}}{\rho_F} \tag{121}$$

Pressure-dependent mass flow (for exponential blowdown):

$$\dot{m}_{F,\text{pressure_dependent}} = \dot{m}_F \sqrt{\frac{P_{u,F}}{P_{\text{ref},F}}} \tag{122}$$

where $P_{\text{ref},F} = 5 \times 10^6$ Pa (725 psi).
This creates exponential decay because:

$$\frac{dP}{dt} = \frac{d}{dt}\left(\frac{mRT}{V}\right) = -\frac{mRT}{V^2}\frac{dV}{dt} = -\frac{P}{V}\frac{\dot{m}_F}{\rho_F} = -kP \tag{123}$$

where $k = \frac{\dot{m}_F}{V\rho_F} \propto \sqrt{P}$, giving $P(t) = P_0 e^{-kt}$.

10.1.5 Tank Gas Mass and Pressure

Gas mass update:

$$m_{\text{gas,F},k+1} = m_{\text{gas,F},k} + \Delta t \cdot \dot{m}_{\text{gas,F}} \tag{124}$$

Temperature (polytropic):

$$T_{\text{gas,F},k+1} = T_{\text{gas,F},0}\left(\frac{m_{\text{gas,F},k+1}/V_{u,F,k+1}}{m_{\text{gas,F},0}/V_{u,F,0}}\right)^{n-1} \tag{125}$$

Pressure:

$$P_{u,F,k+1} = \frac{m_{\text{gas,F},k+1} R T_{\text{gas,F},k+1}}{V_{u,F,k+1} Z} \tag{126}$$

10.2 Startup Behavior

At startup, when flow begins ($\dot{m}_F > 0$):

1. Ullage volume increases: $V_{u,F,k+1} > V_{u,F,k}$
2. If valve closed ($u_F = 0$): gas mass constant, $m_{\text{gas,F},k+1} = m_{\text{gas,F},k}$
3. Pressure drops immediately: $P_{u,F,k+1} = \frac{m_{\text{gas,F},k} RT}{V_{u,F,k+1}} < P_{u,F,k}$
4. Controller hasn't responded yet (catch-up phase)

10.3 End State Behavior

When all propellant is exhausted:

- Ullage volumes approach full tank volumes: $V_{u,F} \to V_{\text{tank,F}}$, $V_{u,O} \to V_{\text{tank,O}}$
- Tank volumes are much larger than COPV: $V_{\text{tank}} \gg V_{\text{copv}}$
- Final pressure: $P_{\text{final}} \approx P_{\text{initial}} \cdot \frac{V_{\text{copv}}}{V_{\text{tank}}} \ll P_{\text{initial}}$
- Example: 2750 psi in 4.5L COPV $\to$ 1100 psi in 11L fuel tank (if all gas goes to one tank)
- With both tanks and growing ullage, final pressure can be 10-20% of initial

11 Complete DDP Algorithm

11.1 Full Algorithm Pseudocode

```
function solve_ddp(x0, u_seq_init, F_ref, MR_ref, cfg, params, engine_wrapper):
    u_seq = u_seq_init
    reg = reg_init

    for iteration = 1 to max_iterations:
        // Forward rollout
        x_seq, eng_estimates, cost, violations = forward_rollout(
            x0, u_seq, F_ref, MR_ref, cfg, params, engine_wrapper
        )

        // Check convergence
        if |cost - prev_cost| < tolerance:
            break

        // Backward pass
        k_seq, K_seq, Vx, Vxx = backward_pass(
            x_seq, u_seq, eng_estimates, F_ref, MR_ref,
            cfg, params, engine_wrapper, reg, w_bar
        )

        // Forward line search
        u_seq_new, alpha, cost_new = forward_line_search(
            x0, u_seq, k_seq, K_seq, x_seq, F_ref, MR_ref,
            cfg, params, engine_wrapper, alpha_init, alpha_min
        )

        // Update
        if cost_new < cost:
            u_seq = u_seq_new
            reg = max(reg / reg_factor, reg_init)
        else:
            reg *= reg_factor
            if reg > 1e6:
                reg = reg_init
                u_seq = u_seq + small_random_perturbation

    return best_solution
```

11.2 Forward Rollout Details

For each time step $k = 0, \ldots, N-1$:

1. Get current state: $\mathbf{x}_k$
2. Estimate engine performance:

$$\text{eng_est}_k = \text{engine_wrapper.estimate_from_pressures}(P_{u,F,k}, P_{u,O,k}) \tag{127}$$

   This returns: F_k, MR_k, $\dot{m}_{F,k}$, $\dot{m}_{O,k}$

3. Compute constraints:

$$\text{constraints}_k = \text{constraint_values}(\mathbf{x}_k, \text{eng_est}_k, \text{cfg}) \tag{128}$$

4. Compute running cost:

$$\ell_k = \text{running_cost}(\mathbf{x}_k, \mathbf{u}_k, \text{eng_est}_k, F_{\text{ref},k}, \text{MR}_{\text{ref},k}, \text{cfg}, \text{constraints}_k, \Delta t) \tag{129}$$

5. Step dynamics:

$$\mathbf{x}_{k+1} = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, \Delta t, \dot{m}_{F,k}, \dot{m}_{O,k}, \text{params}) \tag{130}$$

Total cost:

$$J = \sum_{k=0}^{N-1} \ell_k \tag{131}$$

11.3 Backward Pass Details

Initialize terminal value function:

$$\mathbf{V}_{\mathbf{x},N} = \mathbf{0}, \quad \mathbf{V}_{\mathbf{xx},N} = \mathbf{0} \tag{132}$$

For $k = N-1, \ldots, 0$:

1. Linearize dynamics:

$$\mathbf{A}_k = \frac{\partial \mathbf{f}}{\partial \mathbf{x}}\Big|_{\mathbf{x}_k,\mathbf{u}_k}, \quad \mathbf{B}_k = \frac{\partial \mathbf{f}}{\partial \mathbf{u}}\Big|_{\mathbf{x}_k,\mathbf{u}_k} \tag{133}$$

2. Compute cost derivatives:

$$\ell_{\mathbf{x}}, \ell_{\mathbf{u}}, \ell_{\mathbf{xx}}, \ell_{\mathbf{uu}}, \ell_{\mathbf{ux}} = \text{cost_derivatives}(\mathbf{x}_k, \mathbf{u}_k, \text{eng_est}_k, F_{\text{ref},k}, \text{MR}_{\text{ref},k}, \text{cfg}, \Delta t, \text{engine_wrapper}) \tag{134}$$

3. Compute Q-function derivatives:

$$\mathbf{Q}_{\mathbf{x}} = \ell_{\mathbf{x}} + \mathbf{A}_k^T \mathbf{V}_{\mathbf{x},k+1} \tag{135}$$
$$\mathbf{Q}_{\mathbf{u}} = \ell_{\mathbf{u}} + \mathbf{B}_k^T \mathbf{V}_{\mathbf{x},k+1} \tag{136}$$
$$\mathbf{Q}_{\mathbf{xx}} = \ell_{\mathbf{xx}} + \mathbf{A}_k^T \mathbf{V}_{\mathbf{xx},k+1} \mathbf{A}_k \tag{137}$$
$$\mathbf{Q}_{\mathbf{uu}} = \ell_{\mathbf{uu}} + \mathbf{B}_k^T \mathbf{V}_{\mathbf{xx},k+1} \mathbf{B}_k \tag{138}$$
$$\mathbf{Q}_{\mathbf{ux}} = \ell_{\mathbf{ux}} + \mathbf{B}_k^T \mathbf{V}_{\mathbf{xx},k+1} \mathbf{A}_k \tag{139}$$

4. Robustification:

$$\mathbf{Q}_{\mathbf{xx}} \leftarrow \mathbf{Q}_{\mathbf{xx}} + \gamma \text{diag}(\bar{\mathbf{w}}^2) \tag{140}$$

5. Regularization:

$$\mathbf{Q}_{\mathbf{uu},\text{reg}} = \mathbf{Q}_{\mathbf{uu}} + \lambda \mathbf{I} \tag{141}$$

Ensure positive definite using Cholesky decomposition.

6. Compute gains:

$$\mathbf{k}_k = -(\mathbf{Q}_{\mathbf{uu},\text{reg}})^{-1} \mathbf{Q}_{\mathbf{u}}, \quad \mathbf{K}_k = -(\mathbf{Q}_{\mathbf{uu},\text{reg}})^{-1} \mathbf{Q}_{\mathbf{ux}} \tag{142}$$

Scale gains: $\mathbf{k}_k \leftarrow 2.0 \cdot \mathbf{k}_k$, $\mathbf{K}_k \leftarrow 2.0 \cdot \mathbf{K}_k$

7. Update value function:

$$\mathbf{V}_{\mathbf{x},k} = \mathbf{Q}_{\mathbf{x}} + \mathbf{K}_k^T \mathbf{Q}_{\mathbf{uu},\text{reg}} \mathbf{k}_k + \mathbf{K}_k^T \mathbf{Q}_{\mathbf{u}} + \mathbf{Q}_{\mathbf{ux}}^T \mathbf{k}_k \tag{143}$$
$$\mathbf{V}_{\mathbf{xx},k} = \mathbf{Q}_{\mathbf{xx}} + \mathbf{K}_k^T \mathbf{Q}_{\mathbf{uu},\text{reg}} \mathbf{K}_k + \mathbf{K}_k^T \mathbf{Q}_{\mathbf{ux}} + \mathbf{Q}_{\mathbf{ux}}^T \mathbf{K}_k \tag{144}$$

Project $\mathbf{V}_{\mathbf{xx},k}$ to positive semi-definite.

11.4 Forward Line Search Details

Initialize:

$$\alpha = \min(5.0 \cdot \alpha_{\text{init}}, 5.0) \tag{145}$$

If $J_{\text{nom}} > 10^4$:

$$\alpha = \min(10.0 \cdot \alpha_{\text{init}}, 10.0) \tag{146}$$

While $\alpha \geq \alpha_{\min}$:

1. For each $k = 0, \ldots, N-1$:

$$\delta \mathbf{x}_k = \mathbf{x}_k^{\text{new}} - \mathbf{x}_k^{\text{nom}} \tag{147}$$

$$\delta \mathbf{u}_k = \alpha \mathbf{k}_k + \mathbf{K}_k \delta \mathbf{x}_k \tag{148}$$

   Scale: $\delta \mathbf{u}_k \leftarrow 2.0 \cdot \delta \mathbf{u}_k$

$$\mathbf{u}_k^{\text{new}} = \text{clip}(\mathbf{u}_k^{\text{nom}} + \delta \mathbf{u}_k, 0, 1) \tag{149}$$

2. Forward rollout with new control:

$$\mathbf{x}_{k+1}^{\text{new}} = \mathbf{f}(\mathbf{x}_k^{\text{new}}, \mathbf{u}_k^{\text{new}}, \Delta t, \dot{m}_F, \dot{m}_O) \tag{150}$$

3. Compute cost: J_{new}
4. If $J_{\text{new}} < J_{\text{best}}$:

$$J_{\text{best}} = J_{\text{new}}, \quad \alpha_{\text{best}} = \alpha, \quad \mathbf{u}_{\text{best}} = \mathbf{u}^{\text{new}} \tag{151}$$

   If improvement $> 10^{-6}$: break

5. Reduce step size: $\alpha \leftarrow \alpha/2$

12 Cost Function Detailed Derivation

12.1 Thrust Tracking Cost

The thrust error is:

$$F_{\text{error}} = F_k - F_{\text{ref},k} \tag{152}$$

Normalized to prevent numerical blowup:

$$F_{\text{error,norm}} = \frac{F_{\text{error}}}{\max(|F_{\text{ref},k}|, 1.0)} \tag{153}$$

Cost with asymmetric penalty:

$$\text{Cost}_F = \begin{cases} q_F F_{\text{error,norm}}^2 \cdot 2.0 & \text{if } F_{\text{error}} < 0 \text{ (thrust deficit)} \\ q_F F_{\text{error,norm}}^2 & \text{if } F_{\text{error}} \geq 0 \text{ (thrust excess)} \end{cases} \tag{154}$$

The 2x penalty for thrust deficit strongly incentivizes increasing control when thrust is too low.

12.2 Mixture Ratio Tracking Cost

$$\mathrm{MR}_{\mathrm{error}} = \mathrm{MR}_k - \mathrm{MR}_{\mathrm{ref},k} \tag{155}$$

$$\mathrm{MR}_{\mathrm{error,norm}} = \frac{\mathrm{MR}_{\mathrm{error}}}{\max(|\mathrm{MR}_{\mathrm{ref},k}|, 1.0)} \tag{156}$$

$$\mathrm{Cost}_{\mathrm{MR}} = q_{\mathrm{MR}} \mathrm{MR}^2_{\mathrm{error,norm}} \tag{157}$$

12.3 Gas Consumption Cost

Approximate COPV pressure drop:

$$\Delta P_{\mathrm{copv}} = \Delta t \left(c_F u_F + c_O u_O + \mathrm{loss} \right) \tag{158}$$

where:

- $c_F = 10^5$ Pa/s per unit u_F
- $c_O = 10^5$ Pa/s per unit u_O
- loss $= 10^3$ Pa/s (leakage)

Cost:

$$\mathrm{Cost}_{\mathrm{gas}} = q_{\mathrm{gas}} \Delta P_{\mathrm{copv}} \tag{159}$$

With $q_{\mathrm{gas}} = 0.001$, this is a small penalty to allow controller freedom.

12.4 Constraint Penalties

For each hard constraint violation:

$$\mathrm{Cost}_{\mathrm{constraint}} = w_{\mathrm{constraint}} \left(\frac{\mathrm{violation}}{\max(|\mathrm{violation}|, 1.0)} \right)^2 \tag{160}$$

where $w_{\mathrm{constraint}} = 10^4$ (moderate penalty, normalized to prevent blowup).

13 Linearization via Finite Differences

13.1 State Jacobian $\mathbf{A}_k$

For each state component $i = 0, \ldots, 10$:

$$[\mathbf{A}_k]_{:,i} = \frac{\mathbf{f}(\mathbf{x}_k + \epsilon \mathbf{e}_i, \mathbf{u}_k) - \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)}{\epsilon} \tag{161}$$

where $\epsilon = 10^{-6}$ and $\mathbf{e}_i$ is the i-th unit vector.

13.2 Control Jacobian $\mathbf{B}_k$

For each control component $j = 0, 1$:

$$[\mathbf{B}_k]_{:,j} = \frac{\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k + \epsilon \mathbf{e}_j) - \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)}{\epsilon} \tag{162}$$

where $\mathbf{u}_k + \epsilon \mathbf{e}_j$ is clipped to $[0, 1]$.

13.3 Cost Derivatives

13.3.1 State Gradient $\ell_{\mathbf{x}}$

For each state component i:

$$[\ell_{\mathbf{x}}]_i = \frac{\ell(\mathbf{x}_k + \epsilon \mathbf{e}_i, \mathbf{u}_k) - \ell(\mathbf{x}_k, \mathbf{u}_k)}{\epsilon} \tag{163}$$

If i corresponds to $P_{u,F}$ or $P_{u,O}$, re-estimate engine performance:

$$\text{eng_est}_{\text{pert}} = \text{engine_wrapper.estimate_from_pressures}(P_{u,F,\text{pert}}, P_{u,O,\text{pert}}) \tag{164}$$

13.3.2 Control Gradient $\ell_{\mathbf{u}}$

For each control component j:

$$[\ell_{\mathbf{u}}]_j = \frac{\ell(\mathbf{x}_k, \mathbf{u}_k + \epsilon \mathbf{e}_j) - \ell(\mathbf{x}_k, \mathbf{u}_k)}{\epsilon} \tag{165}$$

13.3.3 State Hessian $\ell_{\mathbf{xx}}$

Diagonal approximation (for efficiency):

$$[\ell_{\mathbf{xx}}]_{i,i} = \frac{\ell(\mathbf{x}_k + \epsilon \mathbf{e}_i, \mathbf{u}_k) - 2\ell(\mathbf{x}_k, \mathbf{u}_k) + \ell(\mathbf{x}_k - \epsilon \mathbf{e}_i, \mathbf{u}_k)}{\epsilon^2} \tag{166}$$

Off-diagonal terms set to zero.

13.3.4 Control Hessian $\ell_{\mathbf{uu}}$

Diagonal approximation:

$$[\ell_{\mathbf{uu}}]_{j,j} = \frac{\ell(\mathbf{x}_k, \mathbf{u}_k + \epsilon \mathbf{e}_j) - 2\ell(\mathbf{x}_k, \mathbf{u}_k) + \ell(\mathbf{x}_k, \mathbf{u}_k - \epsilon \mathbf{e}_j)}{\epsilon^2} \tag{167}$$

13.3.5 Cross-Derivative $\ell_{\mathbf{ux}}$

Set to zero for efficiency (cross-coupling handled through dynamics).

14 Numerical Stability

14.1 Cholesky Decomposition for Positive Definiteness

To check if $\mathbf{Q_{uu}}$ is positive definite:

$$\mathbf{Q_{uu}} = \mathbf{L}\mathbf{L}^T \tag{168}$$

If Cholesky succeeds, matrix is positive definite. If it fails, add regularization:

$$\mathbf{Q_{uu}} \leftarrow \mathbf{Q_{uu}} + \lambda \mathbf{I} \tag{169}$$

Increase λ by factor of 10 until Cholesky succeeds (up to 5 attempts).

14.2 Condition Number Check

Compute condition number:

$$\kappa(\mathbf{Q_{uu}}) = \frac{\sigma_{\max}}{\sigma_{\min}} \tag{170}$$

If $\kappa > 10^{12}$, add extra regularization:

$$\mathbf{Q_{uu}} \leftarrow \mathbf{Q_{uu}} + 0.1\lambda\mathbf{I} \tag{171}$$

14.3 Value Function Hessian Projection

Ensure $\mathbf{V_{xx}}$ is positive semi-definite:

1. Symmetrize: $\mathbf{V_{xx}} \leftarrow \frac{1}{2}(\mathbf{V_{xx}} + \mathbf{V_{xx}}^T)$
2. Try Cholesky: $\mathbf{V_{xx}} + 10^{-8}\mathbf{I} = \mathbf{L}\mathbf{L}^T$
3. If fails, eigendecomposition:

$$\mathbf{V_{xx}} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T \tag{172}$$

   Project negative eigenvalues to zero:

$$\mathbf{\Lambda} \leftarrow \max(\mathbf{\Lambda}, 0) \tag{173}$$

   Reconstruct:

$$\mathbf{V_{xx}} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T \tag{174}$$

4. If eigendecomposition fails, add regularization and use diagonal approximation.

15 Constraint Handling

15.1 Hard Constraints

- **COPV minimum**: $P_{\text{copv}} \geq P_{\text{copv,min}}$ (typically 145 psi = 1 MPa)
- **Ullage maximum**: $V_{u,F} \leq V_{\text{tank,F}}$, $V_{u,O} \leq V_{\text{tank,O}}$
- **Mixture ratio bounds**: $\text{MR}_{\min} \leq \text{MR} \leq \text{MR}_{\max}$ (typically 1.5 to 3.0)
- **Injector pressure drop**: $P_d - P_{\text{chamber}} \geq \text{dp}_{\min}$ (typically 10% of P_d)

15.2 Constraint Violation Computation

For COPV minimum:

$$\text{violation}_{\text{copv}} = P_{\text{copv,min}} - P_{\text{copv}} \tag{175}$$

Positive = violation, negative = satisfied with margin.
For mixture ratio:

$$\text{violation}_{\text{MR,min}} = \text{MR}_{\min} - \text{MR} \tag{176}$$

$$\text{violation}_{\text{MR,max}} = \text{MR} - \text{MR}_{\max} \tag{177}$$

16 Implementation Parameters

16.1 Default Configuration

- Prediction horizon: $N = 50$ steps
- Time step: $\Delta t = 0.01$ s (100 Hz)
- Horizon length: $N\Delta t = 0.5$ s
- Cost weights: $q_F = 100$, $q_{\text{MR}} = 10$, $q_{\text{gas}} = 0.001$, $q_{\text{switch}} = 0.0001$
- Max iterations: 20
- Convergence tolerance: 10^{-3}
- Line search: $\alpha_{\text{init}} = 2.0$, $\alpha_{\text{min}} = 10^{-5}$
- Regularization: $\lambda_{\text{init}} = 10^{-4}$, $\lambda_{\text{factor}} = 5.0$

16.2 Physical Constants

- Gas constant: $R = 296.8$ J/(kg·K) for N_2
- Specific heat ratio: $\gamma = 1.4$ for N_2
- Polytropic exponent: $n = 1.2$ (typical for blowdown)
- Discharge coefficients: $C_d = 0.7$ (regulator), $C_d = 0.65$ (valves)
- Flow areas: $A_{\text{reg}} = 2 \times 10^{-5}$ m^2, $A_{\text{valve}} = 5 \times 10^{-5}$ m^2
- Propellant densities: $\rho_F = 800$ kg/m^3 (RP-1), $\rho_O = 1140$ kg/m^3 (LOX)

17 Common Issues and Troubleshooting

This section addresses common issues that arise when implementing or debugging the controller.

17.1 Controller Not Responding

Symptom: Control inputs don't change, or change very slowly.
Possible causes:

1. **Cost weights too small**: If q_F is too small, controller doesn't care about thrust tracking. **Fix**: Increase q_F.
2. **Regularization too high**: Large λ makes controller conservative. **Fix**: Reduce λ_{init} or check why regularization is increasing.
3. **Gains too small**: If $\mathbf{k}_k$ and $\mathbf{K}_k$ are tiny, control changes are negligible. **Fix**: Check $\mathbf{Q_u}$ - if it's small, the cost gradient is weak.
4. **Line search failing**: If α is always too small, improvements are rejected. **Fix**: Check if cost is actually decreasing in forward rollout.

17.2 Objective Function Blowing Up

Symptom: Cost J becomes very large (e.g., $> 10^6$).
Possible causes:

1. **Unnormalized cost terms**: If thrust error is 1000 N and $q_F = 100$, cost is $100 \times 1000^2 = 10^8$. **Fix**: Normalize errors: $F_{\text{error}}/\max(F_{\text{ref}}, 1)$.

2. **Constraint violations**: Large constraint penalties. **Fix**: Check constraint values, ensure they're reasonable.

3. **Invalid engine estimates**: If engine can't operate (pressures too low), estimates might be invalid. **Fix**: Add checks for valid engine estimates.

17.3 Pressure Not Dropping at Startup

Symptom: When flow starts, pressure doesn't drop immediately.
Possible causes:

1. **Gas mass increasing too fast**: If gas flows into tank faster than ullage grows, pressure increases. **Fix**: Check gas flow rates, ensure they're reasonable.

2. **Ullage volume not updating**: If $V_{u,F}$ doesn't increase when propellant flows, pressure won't drop. **Fix**: Check ullage volume update equation.

3. **Initial conditions wrong**: If initial gas mass is too high, pressure starts too high. **Fix**: Check initialization from ideal gas law.

17.4 DDP Not Converging

Symptom: Cost doesn't decrease, or oscillates.
Possible causes:

1. **Step size too large**: If α is too large, we overshoot the minimum. **Fix**: Reduce α_{init}.

2. **Linearization inaccurate**: If dynamics are highly nonlinear, linearization is poor. **Fix**: Check dynamics model, ensure it's smooth.

3. **Local minimum**: DDP finds local, not global, minimum. **Fix**: Try different initial guesses, or add random perturbations.

17.5 Numerical Instability

Symptom: NaN or Inf values, or eigenvalues not converging.
Possible causes:

1. **Matrix condition number too high**: If $\kappa(\mathbf{Q_{uu}}) > 10^{12}$, inversion is unstable. **Fix**: Add more regularization.

2. **State values extreme**: If pressures are $> 10^8$ Pa or < 0, numerical errors occur. **Fix**: Add bounds checking, clamp values.

3. **Division by zero**: If volumes or masses are zero, divisions fail. **Fix**: Add checks: $V > \epsilon$, $m > \epsilon$.

18 Practical Implementation Tips

18.1 Initialization

State initialization:

- Pressures: From sensors (if available) or from config
- Ullage volumes: From config or estimate from initial propellant mass
- Gas masses: Compute from ideal gas law: $m = PVZ/(RT)$

Control initialization:

- Warm start: Use previous solution (shifted)
- Cold start: Use small constant values (e.g., 0.1)
- Heuristic: Proportional to thrust error

18.2 Parameter Tuning

Cost weights:

- Start with $q_F = 100$, $q_{\text{MR}} = 10$, $q_{\text{gas}} = 0.001$
- If thrust tracking poor: Increase q_F
- If gas consumption high: Increase q_{gas}
- If control chattering: Increase q_{switch}

DDP parameters:

- $\alpha_{\text{init}} = 2.0$: Good starting point
- $\lambda_{\text{init}} = 10^{-4}$: Small regularization
- $N = 50$: Horizon length (balance between computation and prediction)

18.3 Debugging

Print statements:

- Cost at each iteration
- Control values
- State values (especially pressures)
- Engine estimates (thrust, mixture ratio)

Visualization:

- Plot cost vs iteration
- Plot control sequence
- Plot state trajectory
- Plot thrust tracking

18.4 Performance Optimization

Computation time:

- Use diagonal Hessian approximation (faster)
- Reduce horizon length N (if acceptable)
- Reduce max iterations (if converged early)
- Cache engine estimates (if pressures haven't changed)

Memory:

- Store only necessary data (not full trajectory history)
- Use float32 instead of float64 (if acceptable precision)

19 Summary

This framework provides a complete mathematical treatment of robust DDP control for liquid rocket engines, from first principles physics through numerical implementation. The key innovations are:

1. Gas mass tracking for proper mass conservation
2. Polytropic processes for realistic temperature changes
3. Choked/subsonic flow equations for accurate gas flow
4. Normalized cost functions to prevent numerical blowup
5. Robustification for uncertainty handling
6. Hardware-matching oscillations and damping
7. Startup behavior: immediate pressure drop when flow starts
8. End state: final pressure much lower due to volume expansion

The complete system is now ready for implementation and validation against hardware data.